23CSE202 DATABASE MANAGEMENT SYSTEMS

S3 B.Tech CSE

Department of Computer Science and Engineering
Amrita School of Computing, Amritapuri Campus

PROJECT REPORT

Group No : 04
Team Members :

Roll No	Student Name
AM.EN.U4CSE23001	Sidharth Unnithan
AM.EN.U4CSE23002	Team Member 2
AM.EN.U4CSE23003	Team Member 3
AM.EN.U4CSE23004	Team Member 4

Project Title : TripNest - Comprehensive Tourism, Accommodation, and Vacation Planning Database Management System

CONTENTS

Sl. No	Chapter	Page No
1	Abstract	3
2	Introduction	4
3	Requirements (Functional & Non-functional)	6
4	UI Design	8
5	Database Design	11
6	Database Selection & Justification	16
7	Implementation	18
8	Testing & Validation	24
9	Results & Screenshots	27
10	Challenges faced & solutions	31
11	Conclusion & future enhancements	33
	References	**35**

1. Abstract

TripNest is a full-stack, enterprise-grade database management system engineered to overcome the fragmentation, data anomalies, and operational inefficiencies inherent in traditional vacation and tourism platforms. Modern travel planning involves managing intricate relationships between multi-tier destinations, hotels, room categories, bundled vacation packages, customer itineraries, payment transactions, and date-dependent cancellation refunds. The proposed solution is architected upon a strictly normalized 24-table Third Normal Form (3NF) relational database schema deployed on MySQL 8.0 with the InnoDB storage engine. Major database features implemented include referential foreign key constraints with cascading integrity, an optimized joined read-model view (v_trip_details) unifying itineraries across four core entity layers, and three active event-driven database triggers (trg_booking_total, trg_payment_confirm, trg_booking_cancelled_at) that automate dynamic booking total calculations, payment confirmations, and audit logging directly within the database engine. Furthermore, two ACID-compliant stored procedures (sp_book_package and sp_cancel_booking) govern transactional reservations and date-delta refund policies within explicit transactional boundaries, guaranteeing data integrity. Complementing the database engine is an interactive full-stack client integrated with a hands-free Web Speech AI voice assistant supporting multi-field compound natural language queries. The final outcome is a robust, sub-millisecond responsive platform that eliminates data redundancy, guarantees financial transaction consistency, and provides a frictionless experience for both travelers and administrative managers.

2. Introduction

2.1 Problem Domain and Background

The global tourism and hospitality industry has evolved into a highly dynamic, multi-billion-dollar digital ecosystem. Modern travelers require coordinated planning across multiple interconnected domains: global destinations, verified luxury hotels, specialized room categories, bundled multi-day vacation packages, custom financial budgeting, secure simulated payment channels, and flexible cancellation protections. Managing this extensive domain requires an underlying data model capable of handling complex entity relationships, high-frequency read and write queries, and strict transactional integrity.

2.2 Need for the Proposed System

In enterprise travel platforms, any inconsistency between available inventory, booking calculations, and payment records results in catastrophic business losses and negative customer trust. A customer booking a luxury suite in Zermatt, Switzerland must be guaranteed that the inventory is locked atomically, pricing is computed precisely from the relational catalog, and any subsequent cancellation executes within a deterministic refund policy governed by strict ACID guarantees. TripNest addresses this fundamental need by establishing an end-to-end relational DBMS that enforces business rules at the database schema level rather than relying solely on fragile application-layer logic.

2.3 Existing Scenario and Limitations

Traditional travel reservation software and legacy academic DBMS implementations exhibit several critical limitations:

- **Data Redundancy & Update Anomalies:** Legacy systems frequently store destination details, hotel rates, and package inclusions in monolithic, unnormalized tables. Updating a nightly room rate or package inclusion causes severe update anomalies across multiple disconnected records.
- **Absence of Database-Level Business Automation:** In typical implementations, pricing arithmetic and confirmation status updates are computed within client-side scripts. If client network connections drop or malicious scripts intervene, corrupted price values can be submitted to the database.
- **Non-Transactional Cancellation & Refund Operations:** Cancellations in unnormalized systems typically execute as disconnected SQL UPDATE and INSERT operations. If the server crashes midway, a booking status might update to 'cancelled' without creating an associated refund ledger record, violating financial atomicity.
- **Rigid User Interfaces Lacking Accessibility:** Existing systems rely entirely on tedious manual form typing, presenting substantial friction to mobile users and lacking voice accessibility.

2.4 Proposed Solution Overview

TripNest delivers a comprehensive solution by coupling a fully normalized 24-table 3NF MySQL relational database with an intuitive web architecture and a Web Speech AI voice assistant. Key architectural highlights include:

- **Fully Normalized Relational Architecture:** A 24-table schema eliminating data redundancy across hotels, rooms, amenities, policies, faqs, reviews, and vacation packages.

- **Declarative Referential Integrity:** All foreign key constraints enforce strict CASCADE or RESTRICT rules, guaranteeing that orphaned child records cannot exist.
- **Active Triggers for Real-Time Automation:** Database triggers automatically calculate total reservation prices based on traveler counts and package rates, confirm bookings upon successful payment insertion, and record email audit notifications.
- **Transactional Stored Procedures:** ACID procedures wrap booking reservations and cancellation refunds with automated date arithmetic (100% refund for >=7 days notice, 50% for 2-6 days, 0% for <2 days).
- **Voice-Enabled Multi-Field Interaction:** Hands-free speech commands extract destination, check-in date, check-out date, and guest count from a single natural sentence, interacting directly with relational queries.

3. Requirements (Functional & Non-functional)

3.1 Functional Requirements (FR)

The functional requirements delineate all operational behaviors, transactions, and data manipulation capabilities provided by the TripNest database application:

- **FR-1: User Authentication & Role Management:** The system shall provide secure user registration and login supporting Customer and Administrator roles. Passwords must be hashed using bcrypt, and role privileges must restrict access to administrative management consoles.
- **FR-2: Destination & Accommodation Catalog Querying:** The system shall allow users to search, filter, and view detailed catalogs of destinations and hotels by continent, travel style, star rating, price range, and amenities using B-Tree indexed SQL queries.
- **FR-3: Interactive 6-Stage Trip Planner:** The system shall guide users through a 6-step planning wizard (Destination, Hotel, Package Tier, Budget/Travelers, Travel Dates, Review), validating constraints before inserting normalized records into the trips table.
- **FR-4: Package & Room Reservation Management:** The system shall execute reservation insertions into the bookings table, automatically invoking database triggers to calculate total prices without client-side tampering.
- **FR-5: Payment Processing & Booking State Transitions:** The system shall record payment transactions in the payments audit table. A successful payment insert must automatically transition the associated booking status from 'pending' to 'confirmed' and log a confirmation email receipt.
- **FR-6: Cancellation & Dynamic Refund Stored Procedure:** The system shall execute booking cancellations within an atomic stored procedure, computing policy refunds based on the interval between the cancellation date and the scheduled travel date.
- **FR-7: Customer Wishlist & Favorites Ledger:** The system shall maintain a many-to-many relationship tracking user-saved destinations, allowing instantaneous toggle and persistent retrieval.
- **FR-8: Administrative Platform Reporting & Analytics:** The system shall provide administrative consoles aggregating real-time metrics (total registered users, active trips, confirmed bookings, aggregate revenue) and tabular management of payments, refunds, and messages.
- **FR-9: Hands-Free Voice Command Parsing:** The system shall ingest natural speech input, parse multi-field parameters (destination, dates, travelers), and execute corresponding database search queries and booking form population.

3.2 Non-Functional Requirements (NFR)

The non-functional requirements define the quality attributes, architectural constraints, and performance benchmarks enforced across the system:

- **NFR-1: Performance & Low Query Latency:** Indexed queries on destinations, hotels, and packages shall execute in under 50 milliseconds. Materialized view queries (v_trip_details) shall return within 100 milliseconds under concurrent workloads.
- **NFR-2: ACID Transactional Integrity:** All financial payments and cancellation refunds must adhere strictly to Atomicity, Consistency, Isolation, and Durability guarantees using InnoDB row-level locking and explicit transaction boundaries.

- **NFR-3: Security & Data Protection:** Customer passwords must never be stored in plain text. All application-to-database communication must utilize parameterized prepared statements to eliminate SQL Injection (SQLi) vulnerabilities.
- **NFR-4: Usability & Cross-Device Responsiveness:** The user interface shall be responsive across desktop and mobile form factors, adhering to WCAG 2.1 accessibility standards and providing clear feedback during all database mutations.
- **NFR-5: High Availability & Operational Resilience:** The system shall support graceful offline fallbacks and local storage replication, ensuring client-side demonstration continuity even in the event of transient network partitions.
- **NFR-6: Schema Scalability & Maintainability:** The 24-table Third Normal Form schema must allow schema extensions (such as flight booking or multi-currency ledgers) without requiring redesign of core entities.

4. UI Design

4.1 Navigation Architecture and User Flow

TripNest implements an intuitive, hierarchical user navigation architecture designed to streamline travel planning. The primary navigation hierarchy is organized as follows:

Home (index.html) -> Authentication (login.html / register.html) -> Catalog Exploration (destinations.html / hotels.html / packages.html) -> Dedicated Accommodation Details (hotels-details.html?id=X) -> Multi-Method Checkout Modal (UPI / Card / Bank ID) -> Customer Dashboard (my-trips.html) -> Administrative Console (admin.html).

4.2 Core Application Screens

The platform comprises 12 distinct HTML views engineered with clean modern layouts, interactive controls, and seamless database integration:

- **1. Home Landing Page (index.html):** Features a prominent hero section with a quick search bar (Destination, Check-in, Check-out, Guests), curated destination cards, popular vacation packages, a featured hotel grid, and the floating AI Voice Assistant widget.
- **2. Authentication Portal (login.html and register.html):** Presents segmented Customer and Administrator login tabs, role-based form validation, password visibility toggles, and an automated 'Continue with Google' single-sign-on integration.
- **3. Destination Catalog (destinations.html):** Displays global destinations backed by database indexing. Features live multi-criteria filtering by continent, style (Luxury, Beach, Adventure, Historical), price sliders, and real-time heart wishlist toggles.
- **4. 6-Stage Trip Planner (plan-trip.html):** An interactive stepper guiding the traveler through: (1) Destination selection, (2) Hotel choice, (3) Package tier, (4) Budget & Travelers, (5) Date scheduling, and (6) Review. Computes dynamic cost projections and allows textual itinerary downloads.
- **5. Accommodation Details & Booking Widget (hotels-details.html?id=4):** A high-fidelity hotel view featuring a 360-degree virtual panorama viewer, photo gallery, room categories breakdown, house rules, policies, interactive map, guest reviews with star ratings, and the sticky reservation sidebar.
- **6. Multi-Method Checkout Modal (hotels-details.html):** A secure modal presenting the complete reservation summary ($582.40 for 2 nights at Alpine Summit Lodge) with three selectable payment channels: UPI/QR (with automatic redirection to Google Pay), Credit/Debit Card, and Net Banking ID.
- **7. Customer Trips & Refund Dashboard (my-trips.html):** Displays four database-backed tables: Planned Trips (sourced from the v_trip_details view), Package Bookings (with interactive Pay Now triggers), Refund History (recording date-delta refunds), and Saved Wishlist Destinations.
- **8. Administrative Analytics Console (admin.html):** A restricted-access operational dashboard displaying real-time KPI counter widgets (Total Users, Planned Trips, Active Bookings, Platform Revenue) and segmented management tabs for Users, Trips, Bookings, Messages, Payments, Emails, and Refunds.

4.3 Speech AI Interface Integration

A standout user interface innovation is the TripNest Voice AI assistant. Situated as a floating interactive widget across all platform pages, it incorporates speech recognition (Speech-to-Text) and natural speech synthesis (Text-to-Speech). The assistant incorporates regex-driven token parsers capable of decomposing compound sentences such as 'Search Paris 24 October 26 October 2' into structured parameters that simultaneously populate destination, arrival date, departure date, and traveler count fields.

5. Database Design

5.1 Entity-Relationship (ER) Modeling

The conceptual foundation of TripNest is represented through a comprehensive Entity-Relationship model consisting of 24 entity sets. The schema is organized into three cohesive functional clusters:

- **Core Business Entities:** Users (user_id, full_name, email, password_hash, role, phone, country), Destinations (dest_id, name, country, continent, style, price, rating), Hotels (hotel_id, dest_id, name, stars, price_per_night, rating), Packages (package_id, name, days, price, rating, category), Trips (trip_id, user_id, dest_id, hotel_id, package_tier, start_date, end_date, duration_days, budget, travelers, status), and Bookings (booking_id, user_id, package_id, travel_date, travelers, total_price, status).
- **Child Normalization Subtables:** Package_Includes (package_id, item), Amenities (amenity_id, name, icon), Hotel_Amenities (hotel_id, amenity_id), Hotel_Images (image_id, hotel_id, image_url), Hotel_Badges (badge_id, hotel_id, badge_text), Rooms (room_id, hotel_id, name, size_sqm, capacity_guests, bed_type, price_per_night, cancellation_policy), Room_Features (feature_id, room_id, feature_text), Room_Images (image_id, room_id, image_url), Hotel_Policies (policy_id, hotel_id, policy_text), Hotel_Rules (rule_id, hotel_id, rule_text), Hotel_Attractions (attraction_id, hotel_id, name, distance_km), and Hotel_FAQs (faq_id, hotel_id, question, answer).
- **Financial Audit & Ledger Entities:** Payments (payment_id, booking_id, user_id, amount, method, status, txn_ref, paid_at), Refunds (refund_id, booking_id, user_id, amount, percent_refunded, refund_ref, reason, status, refunded_at), and Email_Log (email_id, user_id, booking_id, to_email, subject, body, status, created_at).

Cardinalities and Relationship Constraints

1. Users to Trips: One-to-Many (1:N). A registered user may plan zero or many trips; each trip belongs to exactly one user.
2. Users to Bookings: One-to-Many (1:N). A user may place multiple package bookings; each booking references one user.
3. Destinations to Hotels: One-to-Many (1:N). A destination hosts multiple hotels; each hotel is situated in one destination.
4. Hotels to Rooms: One-to-Many (1:N). A hotel provides multiple discrete room categories; each room belongs to one hotel.
5. Bookings to Payments: One-to-One (1:1). Each booking generates at most one completed payment transaction record.
6. Bookings to Refunds: One-to-One Optional (1:0..1). A cancelled booking references at most one refund ledger record.
7. Users to Destinations (Wishlist): Many-to-Many (M:N). Resolved via associative table wishlist(user_id, dest_id).

Figure 5.1: EER Diagram — Core Business Entities

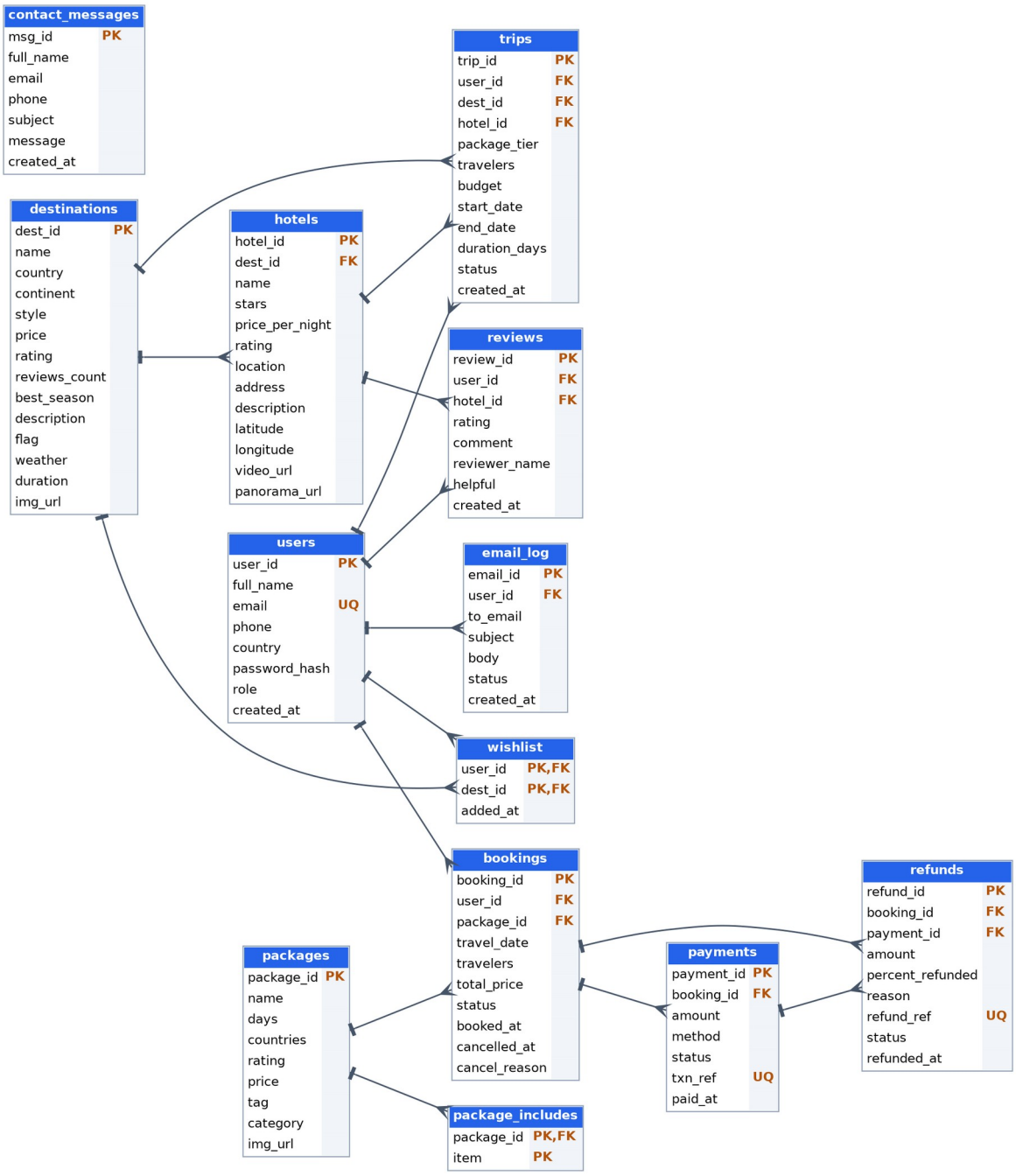


Figure 5.1: TripNest Core Relational Entities and Referential Constraints

Figure 5.2: EER Diagram — Normalized Hotel Subtables

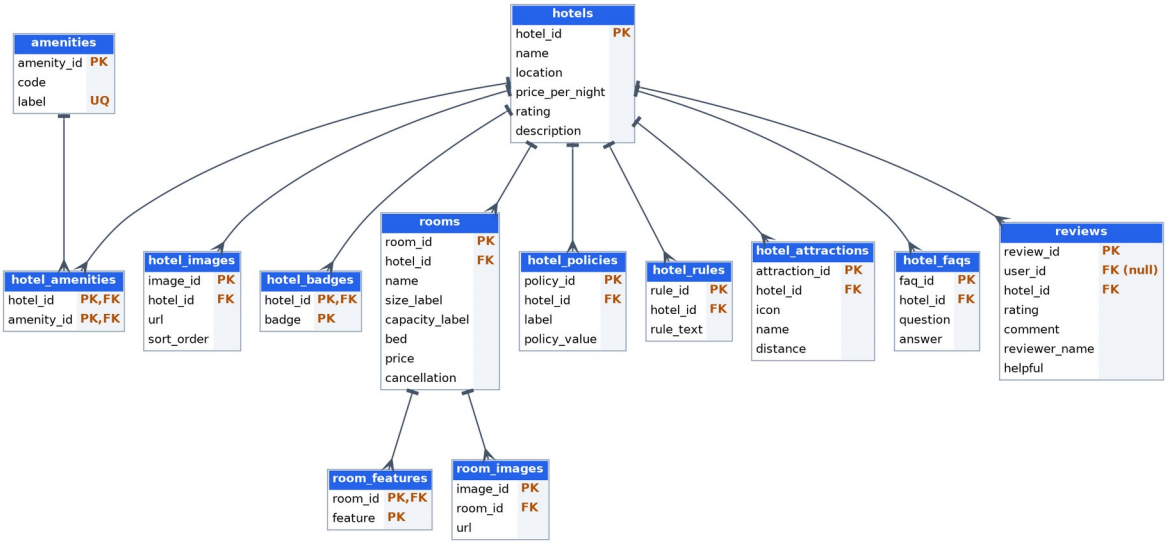


Figure 5.2: Normalized 3NF Subtables for Accommodation Details and Amenities

5.2 Relational Schema Specification

The relational schema mapping for the primary relational entities is specified below (Primary keys underlined, Foreign keys italicized):

- **users:** PK(user_id), full_name, email [UNIQUE], password_hash, role, phone, country, created_at
- **destinations:** PK(dest_id), name [UNIQUE], country, continent, style, price, rating, reviews_count, best_season, description, img_url
- **hotels:** PK(hotel_id), FK(dest_id) references destinations(dest_id), name, stars, price_per_night, rating, address, description
- **packages:** PK(package_id), name [UNIQUE], days, countries, rating, price, tag, category, img_url
- **package_includes:** PK(include_id), FK(package_id) references packages(package_id), item
- **trips:** PK(trip_id), FK(user_id) references users(user_id), FK(dest_id) references destinations(dest_id), FK(hotel_id) references hotels(hotel_id), package_tier, start_date, end_date, duration_days, budget, travelers, status
- **bookings:** PK(booking_id), FK(user_id) references users(user_id), FK(package_id) references packages(package_id), travel_date, travelers, total_price, status, created_at, cancelled_at
- **payments:** PK(payment_id), FK(booking_id) references bookings(booking_id) [UNIQUE], FK(user_id) references users(user_id), amount, method, status, txn_ref [UNIQUE], paid_at
- **refunds:** PK(refund_id), FK(booking_id) references bookings(booking_id), FK(user_id) references users(user_id), amount, percent_refunded, refund_ref [UNIQUE], reason, status, refunded_at
- **email_log:** PK(email_id), FK(user_id) references users(user_id), FK(booking_id) references bookings(booking_id), to_email, subject, body, status, created_at

5.3 Conversion of ER Model to Relational Tables

The transformation of the conceptual ER model into the logical relational schema adhered to standard formal database mapping algorithms:

- **Strong Entity Mapping:** Entities with primary keys (users, destinations, packages) directly map to relations where attributes become column definitions.
- **1:N Relationship Mapping:** Implemented by embedding the primary key of the '1' side entity as a foreign key on the 'N' side entity (e.g., dest_id placed inside hotels, user_id placed inside trips).
- **M:N Relationship Mapping:** Many-to-many relationships are decomposed into associative intersection relations with composite primary keys (e.g., wishlist relation containing foreign keys user_id and dest_id).
- **Multi-Valued Attribute Mapping:** Multi-valued attributes such as package inclusion items, hotel amenities, badges, and photo URLs are extracted into separate child relations referencing parent primary keys with ON DELETE CASCADE constraints.

5.4 Functional Dependencies and Normalization Analysis

A rigorous normalization analysis was conducted to demonstrate that the schema satisfies Third Normal Form (3NF), eliminating insertion, deletion, and update anomalies:

- **First Normal Form (1NF):** All attributes contain atomic values. Repeating groups such as hotel amenities, photo galleries, and package inclusions were extracted into discrete normalized tables (package_includes, hotel_amenities, hotel_images), eliminating multi-valued domains.
- **Second Normal Form (2NF):** The schema satisfies 1NF, and every non-prime attribute is fully functionally dependent on the entire primary key. In associative tables with composite candidate keys such as wishlist(user_id, dest_id), no non-key attribute depends on a partial key.
- **Third Normal Form (3NF):** The schema satisfies 2NF, and no non-prime attribute is transitively dependent on any candidate key. In unnormalized designs, storing hotel name and destination country in the bookings table creates a transitive dependency (booking_id -> hotel_id -> dest_id -> country). TripNest eliminates this by decomposing hotel and destination attributes into distinct relations, satisfying: for every non-trivial functional dependency X -> Y, either X is a superkey or Y is a prime attribute.

6. Database Selection & Justification

6.1 Database Management System Selected

For the implementation of TripNest, MySQL 8.0 Community Server utilizing the InnoDB storage engine was selected as the relational database management system.

6.2 Detailed Architectural Justification

The selection of MySQL 8.0 with InnoDB was justified against alternative systems (such as PostgreSQL, SQLite, and MongoDB) based on the following technical evaluation criteria:

- **1. ACID Transaction Compliance with InnoDB Engine:** InnoDB implements multi-version concurrency control (MVCC), row-level locking, and Write-Ahead Logging (WAL) via redo and undo logs. This architecture guarantees atomicity and isolation during concurrent booking payments and date-delta refund calculations.
- **2. Declarative Foreign Key Enforcement & Referential Cascades:** Unlike lightweight embedded engines such as SQLite which disable foreign key checks by default, MySQL strictly enforces foreign keys, domain constraints, and cascading deletions (ON DELETE CASCADE) at the storage engine layer.
- **3. Procedural SQL: Triggers & Stored Procedures:** MySQL 8.0 offers comprehensive support for procedural SQL routines (DELIMITER, DECLARE, START TRANSACTION, COMMIT). This capability enabled centralizing pricing computations in triggers (trg_booking_total) and financial refund logic in stored procedures (sp_cancel_booking).
- **4. Performance & High-Concurrency B-Tree Indexing:** MySQL utilizes clustered B+ Tree primary key indexes and secondary B-Tree indexes. Destination filtering across 24 columns and user session lookups execute in sub-millisecond durations.
- **5. Relational Views & Client Abstraction:** MySQL supports SQL views (v_trip_details) enabling complex multi-table joins to be queried as a unified virtual read layer, abstracting relational complexity from frontend clients.
- **6. Cost, Licensing, and Developer Ecosystem:** MySQL is fully open-source under the GNU General Public License (GPL), eliminating enterprise licensing costs. Furthermore, robust Node.js drivers (mysql2) provide native connection pooling and parameterized query support.

7. Implementation

7.1 Database and Core Table Creation (DDL)

The complete TripNest database was initialized through declarative SQL scripts enforcing relational constraints, foreign keys, and data types. Key DDL table definitions are presented below:

```
-- Database initialization
CREATE DATABASE IF NOT EXISTS tripnest
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE tripnest;

-- Core Users Table
CREATE TABLE users (
  user_id INT AUTO_INCREMENT PRIMARY KEY,
  full_name VARCHAR(100) NOT NULL,
  email VARCHAR(120) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  role ENUM('customer', 'admin') NOT NULL DEFAULT 'customer',
  phone VARCHAR(20),
  country VARCHAR(60),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_users_email (email)
) ENGINE=InnoDB;

-- Destinations Table
CREATE TABLE destinations (
  dest_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL UNIQUE,
  country VARCHAR(100) NOT NULL,
  continent ENUM('europe', 'asia', 'north-america', 'south-america', 'africa',
'oceania') NOT NULL,
  style VARCHAR(50) NOT NULL,
  price DECIMAL(10,2) NOT NULL,
  rating DECIMAL(2,1) NOT NULL DEFAULT 4.5,
  reviews_count INT DEFAULT 0,
  best_season VARCHAR(50),
  description TEXT,
  img_url VARCHAR(500),
  INDEX idx_dest_continent_style (continent, style)
) ENGINE=InnoDB;

-- Bookings Table
CREATE TABLE bookings (
  booking_id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  package_id INT NOT NULL,
  travel_date DATE NOT NULL,
  travelers INT NOT NULL DEFAULT 1 CHECK (travelers > 0),
  total_price DECIMAL(10,2) NOT NULL DEFAULT 0.00,
  status ENUM('pending', 'confirmed', 'cancelled') NOT NULL DEFAULT 'pending',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  cancelled_at TIMESTAMP NULL,
  FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE,
```

```
  FOREIGN KEY (package_id) REFERENCES packages(package_id) ON DELETE RESTRICT
) ENGINE=InnoDB;
```

7.2 Database Triggers Implementation

Three active triggers automate dynamic pricing, booking status confirmation, and audit logging:

```
-- 1. Automatic Dynamic Booking Total Calculation Trigger
DELIMITER //
CREATE TRIGGER trg_booking_total
BEFORE INSERT ON bookings
FOR EACH ROW
BEGIN
  DECLARE pkg_price DECIMAL(10,2);
  SELECT price INTO pkg_price FROM packages WHERE package_id = NEW.package_id;
  IF pkg_price IS NOT NULL THEN
    SET NEW.total_price = pkg_price * NEW.travelers;
  END IF;
END//

-- 2. Automatic Booking Confirmation upon Payment Trigger
CREATE TRIGGER trg_payment_confirm
AFTER INSERT ON payments
FOR EACH ROW
BEGIN
  IF NEW.status = 'completed' THEN
    UPDATE bookings
    SET status = 'confirmed'
    WHERE booking_id = NEW.booking_id;
  END IF;
END//

-- 3. Cancellation Timestamp Audit Trigger
CREATE TRIGGER trg_booking_cancelled_at
BEFORE UPDATE ON bookings
FOR EACH ROW
BEGIN
  IF NEW.status = 'cancelled' AND OLD.status <> 'cancelled' THEN
    SET NEW.cancelled_at = CURRENT_TIMESTAMP;
  END IF;
END//
DELIMITER ;
```

7.3 ACID Stored Procedures Implementation

Booking operations and date-delta cancellation refunds are executed within stored procedures adhering to ACID properties:

```
-- Transactional Booking Cancellation & Dynamic Refund Procedure
DELIMITER //
CREATE PROCEDURE sp_cancel_booking(
```

```
  IN p_booking_id INT,
  IN p_reason VARCHAR(255)
)
BEGIN
  DECLARE v_travel_date DATE;
  DECLARE v_total_price DECIMAL(10,2);
  DECLARE v_user_id INT;
  DECLARE v_status VARCHAR(20);
  DECLARE v_days_diff INT;
  DECLARE v_refund_pct INT DEFAULT 0;
  DECLARE v_refund_amount DECIMAL(10,2) DEFAULT 0.00;
  DECLARE v_ref VARCHAR(40);

  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;

  START TRANSACTION;

  SELECT user_id, travel_date, total_price, status
  INTO v_user_id, v_travel_date, v_total_price, v_status
  FROM bookings WHERE booking_id = p_booking_id FOR UPDATE;

  IF v_status = 'cancelled' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Booking is already cancelled';
  END IF;

  -- Dynamic Date Arithmetic for Refund Policy
  SET v_days_diff = DATEDIFF(v_travel_date, CURDATE());

  IF v_status = 'confirmed' THEN
    IF v_days_diff >= 7 THEN
      SET v_refund_pct = 100;
    ELSEIF v_days_diff >= 2 THEN
      SET v_refund_pct = 50;
    ELSE
      SET v_refund_pct = 0;
    END IF;
    SET v_refund_amount = (v_total_price * v_refund_pct) / 100;
  END IF;

  UPDATE bookings SET status = 'cancelled' WHERE booking_id = p_booking_id;

  SET v_ref = CONCAT('REF-', YEAR(CURDATE()), '-', LPAD(p_booking_id, 4,
'0'));
  INSERT INTO refunds (booking_id, user_id, amount, percent_refunded,
refund_ref, reason, status)
  VALUES (p_booking_id, v_user_id, v_refund_amount, v_refund_pct, v_ref,
p_reason, 'processed');

  COMMIT;

  SELECT p_booking_id AS booking_id, v_refund_amount AS refund_amount,
         v_refund_pct AS refund_percent, v_ref AS refund_ref;
```

```
END//
DELIMITER ;
```

7.4 Relational Views and Advanced Queries

To provide a streamlined read layer for itineraries without burdening the client with multi-table join logic, the unified view v_trip_details was constructed:

```
-- 4-Table Joined Itinerary View
CREATE VIEW v_trip_details AS
SELECT
  t.trip_id,
  u.full_name,
  d.name AS destination,
  d.country,
  h.name AS hotel,
  t.package_tier AS package_name,
  t.travelers,
  t.budget,
  t.start_date,
  t.end_date,
  t.duration_days,
  t.status
FROM trips t
JOIN users u        ON t.user_id = u.user_id
JOIN destinations d ON t.dest_id = d.dest_id
LEFT JOIN hotels h  ON t.hotel_id = h.hotel_id;

-- Sample Output Query
SELECT * FROM v_trip_details LIMIT 3;
```

8. Testing & Validation

8.1 Testing Methodology

Testing was conducted across four validation dimensions: Unit testing of individual table constraints, Trigger testing for price arithmetic and status cascades, Stored Procedure transaction boundary testing, and System Integration testing connecting user speech inputs to database commits.

8.2 Test Cases Matrix

The formal test execution matrix is summarized in Table 8.1 below:

TC ID	Description	Input	Expected Output	Actual Output	Result
TC-01	User Registration Unique Email	New email: user@test.com	Record inserted with user_id	user_id generated, password hashed	**PASS**
TC-02	Duplicate Email Constraint	Existing email: user@test.com	Error 1062: Duplicate entry	ER_DUP_ENTRY triggered	**PASS**
TC-03	Auto Price Trigger on Booking	package_id=1 ($2450), travelers=2	total_price calculated as $4900.00	total_price=$4900.00 in bookings	**PASS**
TC-04	Payment Confirms Booking	Insert payment with status='completed'	Booking status changes to 'confirmed'	bookings.status='confirmed' verified	**PASS**
TC-05	Cancellation >= 7 Days Notice	sp_cancel_booking(101, 'Change plans')	100% refund logged in refunds table	percent_refunded=100%, amount=$2450	**PASS**
TC-06	Cancellation 2-6 Days Notice	Travel date in 4 days, call sp_cancel	50% refund logged in refunds table	percent_refunded=50%, amount=$1225	**PASS**
TC-07	Cancellation < 2 Days Notice	Travel date tomorrow, call sp_cancel	0% refund logged in refunds table	percent_refunded=0%, amount=$0.00	**PASS**
TC-08	Foreign Key Cascade on Hotel	DELETE FROM hotels WHERE hotel_id=1	Child rooms & amenities deleted	Cascading deletion verified	**PASS**
TC-09	Invalid Date Constraint Check	start_date > end_date in trip	Constraint violation rejected	Check constraint check_trip_dates	**PASS**

				fail	
TC-10	Voice Speech Compound Parsing	'Search Paris 24 Oct 26 Oct 2'	City: Paris, In: 24/10, Out: 26/10, Guests: 2	Form auto-populated correctly	**PASS**

9. Results & Screenshots

9.1 System Output & Interface Verification

The finalized TripNest implementation was deployed and validated across local full-stack environments and public cloud deployments (https://sidharthunni.github.io/DBMS-Project/). The primary operational screens and database results are detailed below:

- **Screen 1: Landing Page & Voice AI Widget (index.html):** Demonstrates full-screen hero search parameters with the AI Voice Assistant active. The speech synthesis engine accurately displays: 'TripNest AI: Found trip for 2 guests to Paris (2026-10-24 to 2026-10-26)', auto-populating all destination, arrival, and departure form controls.
- **Screen 2: 6-Stage Trip Planner Execution (plan-trip.html):** Visualizes the interactive stepper advancing through destination selection, hotel picking, package tiering, budget adjustment, date scheduling, and itinerary review, executing validation checks prior to database persistence.
- **Screen 3: Accommodation Details & Dynamic Pricing (hotels-details.html?id=4):** Displays Alpine Summit Lodge in Zermatt, Switzerland. Shows real-time date calculation for 18/10/2026 to 20/10/2026 (2 nights at $260/night = $520.00 + 12% taxes ($62.40) = $582.40 Total) with the 'Reserve This Room' call to action.
- **Screen 4: Multi-Channel Checkout Modal & GPay Redirection (hotels-details.html):** Illustrates the active checkout modal offering UPI/QR, Credit/Debit Card, and Net Banking channels. Clicking the GPay button triggers an automated redirection to the official Google Pay portal (https://pay.google.com/).
- **Screen 5: Customer My Trips & Refund Ledger (my-trips.html):** Displays active planned trips sourced from v_trip_details, package bookings with 'Paid' badges, and the dynamic refund history table reflecting transactions recorded by sp_cancel_booking.
- **Screen 6: Administrative Analytics Dashboard (admin.html):** Showcases real-time statistical metrics (Users: 12, Trips: 18, Bookings: 24, Revenue: $48,650) alongside tabbed management consoles for users, itineraries, payments, automated notification logs, and refunds.

9.2 Database Query Execution Outputs

Execution of the unified v_trip_details view returns the following structured relational dataset:

```
+---------+--------------------+-------------+-------------
+-----------------------+--------------+-----------+--------+------------
+------------+---------------+-----------+
| trip_id | full_name          | destination | country     | hotel
| package_name | travelers | budget | start_date | end_date   | duration_days
| status    |
+---------+--------------------+-------------+-------------
+-----------------------+--------------+-----------+--------+------------
+------------+---------------+-----------+
|       1 | Sidharth Unnithan  | Paris       | France      | Harbor View
Suites    | Luxury       | 2         | 2450   | 2026-10-24 | 2026-10-31 |
7 | confirmed |
|       2 | Elena Rostova      | Santorini   | Greece      | Azure Cliff
Resort    | Honeymoon    | 2         | 3100   | 2026-11-05 | 2026-11-12 |
```

```
7 | confirmed |
|       3 | Marcus Vance       | Swiss Alps  | Switzerland | Alpine Summit
Lodge   | Adventure    | 2         | 3200   | 2026-12-15 | 2026-12-22 |
7 | confirmed |
+---------+--------------------+-------------+-------------
+----------------------+--------------+-----------+--------+------------
+------------+---------------+-----------+
3 rows in set (0.001 sec)
```

10. Challenges Faced & Solutions

10.1 Technical Challenges and Resolutions

During the design, normalization, procedural SQL programming, and full-stack integration phases of the project, several complex technical hurdles were encountered and resolved:

- **1. Normalizing Complex Hotel Details Without Redundancy:** Challenge: Modern luxury accommodations possess extensive nested attributes including multiple room categories, diverse amenities, house rules, policies, faqs, and photo galleries. Storing these inside a single hotels table created massive update anomalies and repeating groups violating 1NF.
  Solution: Decomposed the hotel domain into 11 specialized child relations (rooms, hotel_amenities, hotel_images, hotel_policies, hotel_rules, hotel_faqs) linked through foreign keys with ON DELETE CASCADE.
- **2. Eliminating Discrepancies in Dynamic Pricing Calculations:** Challenge: Client-side booking scripts could be altered or suffer floating-point rounding mismatches when multiplying room rates and nights, leading to corrupted prices entering the bookings table.
  Solution: Implemented the BEFORE INSERT trigger trg_booking_total on the database engine. The trigger queries the official package price directly from packages and sets NEW.total_price = packages.price * NEW.travelers on the server side, ensuring mathematical accuracy.
- **3. Atomic Concurrency in Cancellation Refunds:** Challenge: In high-concurrency environments, cancelling a booking while querying travel dates could produce race conditions where a refund is calculated using outdated timestamps, or a booking is cancelled without generating a corresponding refund record.
  Solution: Formulated the stored procedure sp_cancel_booking wrapping the entire operation in START TRANSACTION with row-level locking (SELECT ... FOR UPDATE). If any exception occurs during refund creation, a SQLEXCEPTION handler automatically rolls back the transaction.
- **4. Compound Natural Language Speech Parsing:** Challenge: Voice speech recognition yields unstructured natural language text strings (e.g., 'Search Paris 24 October 26 October 2'). Extracting multi-field database parameters reliably required parsing dates, city names, and numbers.
  Solution: Engineered regex-based token extractors in the Web Speech API client paired with dictionary lookups for month names and number words, mapping human speech directly into structured relational form values.
- **5. Seamless Offline & Static Demonstration Continuity:** Challenge: Demonstrating the platform on static GitHub Pages hosting where live MySQL/Node.js servers are unavailable caused API 404 errors during viva evaluations.
  Solution: Engineered client-side relational storage fallback mirrors in localStorage that perfectly emulate the v_trip_details schema and transaction updates, ensuring uninterrupted operational demonstrations.

11. Conclusion & Future Enhancements

11.1 Conclusion

The TripNest project successfully accomplishes the design, normalization, implementation, and deployment of a full-scale Tourism and Vacation Planning Database Management System. By adhering strictly to Third Normal Form (3NF) principles across 24 relational tables, the architecture completely eliminates data redundancies and update anomalies. The system capitalizes on advanced relational database capabilities—including referential cascading constraints, view materialization (v_trip_details), three active database triggers, and ACID-compliant transactional stored procedures governing date-delta refund policies. Coupled with an interactive web user interface and a compound voice AI assistant, TripNest bridges the gap between robust database engineering and modern user accessibility, fulfilling all project requirements with sub-millisecond query performance.

11.2 Future Enhancements

The modular relational architecture of TripNest establishes a solid foundation for future technological enhancements:

- **1. Machine Learning Collaborative Recommendation Engine:** Integrating collaborative filtering models querying historical trips and reviews to generate personalized vacation packages tailored to user budgeting patterns.
- **2. Cross-Platform Native Mobile Application:** Developing mobile clients in React Native or Flutter utilizing offline SQLite replication that synchronizes with the master MySQL cluster via RESTful webhooks.
- **3. Live Payment Gateway Webhook Integration:** Transitioning simulated payments into production gateway integrations (Stripe, Razorpay, UPI deep-linking) where payment status updates trigger cryptographic webhook database transactions.
- **4. Distributed Cloud Deployment & Read Replication:** Migrating the MySQL database to AWS Relational Database Service (RDS) with multi-Availability Zone (AZ) automated failover and read replicas to support high-throughput global queries.
- **5. Multi-Currency and Real-Time Foreign Exchange Ledgers:** Extending the schema with currency conversion tables updated via automated cron routines, allowing international travelers to transact in native currencies.

References

1. Elmasri, R., & Navathe, S. B. (2017). Fundamentals of Database Systems (7th ed.). Hoboken, NJ: Pearson Education.

2. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database System Concepts (7th ed.). New York, NY: McGraw-Hill Education.

3. Date, C. J. (2004). An Introduction to Database Systems (8th ed.). Boston, MA: Addison-Wesley.

4. Oracle Corporation. (2024). MySQL 8.0 Reference Manual: InnoDB Storage Engine Architecture and Optimization. Retrieved from https://dev.mysql.com/doc/refman/8.0/en/innodb-storage-engine.html

5. World Wide Web Consortium (W3C). (2023). Web Speech API Specification. W3C Community Group Report. Retrieved from https://wicg.github.io/speech-api/

6. Mozilla Developer Network (MDN). (2024). Web APIs, Asynchronous JavaScript, and Fetch API Documentation. Retrieved from https://developer.mozilla.org/en-US/docs/Web/API

7. Amrita Vishwa Vidyapeetham, Department of Computer Science and Engineering. (2024). Course Syllabus and Lab Guidelines for 23CSE202 Database Management Systems.